

NSGA-II Multi-Objective Evolutionary Algorithm Formulation for Rolling Stock Circulation

Quantum Annealing-Based Rolling Stock Scheduling Research Suite

September 22, 2026

Abstract

This document outlines the multi-objective mathematical formulation and algorithmic architecture of the Non-dominated Sorting Genetic Algorithm II (NSGA-II) applied to the Rolling Stock Scheduling and Circulation Problem. We formally define Pareto dominance, the 3-objective formulation (f_1, f_2, f_3) , Deb et al.'s Fast Non-Dominated Sorting algorithm, and the crowding distance metric for diversity preservation.

1 Introduction

Real-world rolling stock circulation involves conflicting operational trade-offs that cannot be accurately captured by an ad-hoc single scalarized objective. NSGA-II provides an elitist multi-objective evolutionary framework capable of discovering a diverse Pareto-optimal front of compromise circulation schedules without requiring prior scalarization weights.

2 Multi-Objective Problem Formulation

Let $\mathbf{x} \in \Omega$ denote a candidate schedule chromosome encoding unit assignments across $N = |T|$ timetable trips. NSGA-II simultaneously minimizes three competing objective functions:

$$\min_{\mathbf{x} \in \Omega} \mathbf{F}(\mathbf{x}) = \begin{pmatrix} f_1(\mathbf{x}) \\ f_2(\mathbf{x}) \\ f_3(\mathbf{x}) \end{pmatrix} \quad (1)$$

where:

$$f_1(\mathbf{x}) = \text{deadhead_km}(\mathbf{x}) \quad (\text{Total empty repositioning distance}) \quad (2)$$

$$f_2(\mathbf{x}) = \text{units_used}(\mathbf{x}) \quad (\text{Fleet size / capital trainset deployment}) \quad (3)$$

$$f_3(\mathbf{x}) = \text{uncovered_demand}(\mathbf{x}) + 100 \cdot \text{timing_violations}(\mathbf{x}) \quad (\text{Service quality \& constraint penalty}) \quad (4)$$

3 Pareto Optimality Definitions

3.1 Pareto Dominance

A solution $\mathbf{x}_1 \in \Omega$ is said to **dominate** solution $\mathbf{x}_2 \in \Omega$ (denoted $\mathbf{x}_1 \prec \mathbf{x}_2$) if and only if:

$$\forall m \in \{1, 2, 3\}, \quad f_m(\mathbf{x}_1) \leq f_m(\mathbf{x}_2) \quad \text{and} \quad \exists m \in \{1, 2, 3\}, \quad f_m(\mathbf{x}_1) < f_m(\mathbf{x}_2) \quad (5)$$

3.2 Pareto Optimality and Pareto Front

A solution $\mathbf{x}^* \in \Omega$ is **Pareto optimal** if there exists no other feasible solution $\mathbf{x} \in \Omega$ such that $\mathbf{x} \prec \mathbf{x}^*$.

The set of all Pareto optimal decision vectors is the **Pareto Optimal Set** $\mathcal{P}^*$:

$$\mathcal{P}^* = \{\mathbf{x}^* \in \Omega \mid \nexists \mathbf{x} \in \Omega \text{ such that } \mathbf{x} \prec \mathbf{x}^*\} \quad (6)$$

The image of $\mathcal{P}^*$ in objective space constitutes the **Pareto Front** $\mathcal{PF}^*$:

$$\mathcal{PF}^* = \{\mathbf{F}(\mathbf{x}^*) \in \mathbb{R}^3 \mid \mathbf{x}^* \in \mathcal{P}^*\} \quad (7)$$

4 Fast Non-Dominated Sorting (Deb et al. 2002)

The fast non-dominated sorting procedure partitions a population $\mathcal{R}$ of size $2N$ into hierarchical Pareto fronts $\mathcal{F} = (\mathcal{F}_1, \mathcal{F}_2, \dots, \mathcal{F}_K)$ in $\mathcal{O}(MN^2)$ operations, where $M = 3$ is the number of objectives.

For each individual $p \in \mathcal{R}$, two entities are maintained:

1. S_p : The set of individuals that individual p dominates.
2. n_p : The domination count (number of individuals that dominate individual p).

Algorithm 1 Fast Non-Dominated Sort (Deb et al. 2002)

Require: Population $\mathcal{R}$ of size $2N$

Ensure: Partitioned Pareto fronts $\mathcal{F}_1, \mathcal{F}_2, \dots$

```

1: for each  $p \in \mathcal{R}$  do
2:    $S_p \leftarrow \emptyset, n_p \leftarrow 0$ 
3:   for each  $q \in \mathcal{R}$  do
4:     if  $p \prec q$  then
5:        $S_p \leftarrow S_p \cup \{q\}$ 
6:     else if  $q \prec p$  then
7:        $n_p \leftarrow n_p + 1$ 
8:     end if
9:   end for
10:  if  $n_p = 0$  then
11:     $p_{\text{rank}} \leftarrow 1$ , Add  $p$  to  $\mathcal{F}_1$ 
12:  end if
13: end for
14:  $i \leftarrow 1$ 
15: while  $\mathcal{F}_i \neq \emptyset$  do
16:    $\mathcal{H} \leftarrow \emptyset$ 
17:   for each  $p \in \mathcal{F}_i$  do
18:     for each  $q \in S_p$  do
19:        $n_q \leftarrow n_q - 1$ 
20:       if  $n_q = 0$  then
21:          $q_{\text{rank}} \leftarrow i + 1$ , Add  $q$  to  $\mathcal{H}$ 
22:       end if
23:     end for
24:   end for
25:    $i \leftarrow i + 1, \mathcal{F}_i \leftarrow \mathcal{H}$ 
26: end while
27: return  $\mathcal{F}_1, \mathcal{F}_2, \dots$ 

```

5 Crowding Distance Metric

To preserve population diversity along each front without specifying niching parameters, NSGA-II assigns a crowding distance metric d_i representing the density of solutions surrounding individual i .

For a non-dominated front $\mathcal{I} = \{1, 2, \dots, l\}$:

1. Initialize distance for all solutions: $d_i \leftarrow 0$ for all $i \in \mathcal{I}$.
2. For each objective dimension $m \in \{1, 2, 3\}$:
 - Sort front $\mathcal{I}$ in ascending order based on objective value f_m .
 - Assign infinite distance to the boundary solutions:

$$d_{\mathcal{I}[1]} \leftarrow \infty, \quad d_{\mathcal{I}[l]} \leftarrow \infty \quad (8)$$

- For intermediate indices $j \in \{2, \dots, l-1\}$, update crowding distance:

$$d_{\mathcal{I}[j]} \leftarrow d_{\mathcal{I}[j]} + \frac{f_m(\mathcal{I}[j+1]) - f_m(\mathcal{I}[j-1])}{f_m^{\max} - f_m^{\min}} \quad (9)$$

where $f_m^{\max}$ and $f_m^{\min}$ denote the maximum and minimum values of objective m in front $\mathcal{I}$.

6 Crowded Tournament Selection and Elitist Loop

During binary tournament selection, individual i is preferred over individual j (denoted $i \prec_n j$) using the crowded comparison operator:

$$i \prec_n j \iff (r_i < r_j) \vee (r_i = r_j \wedge d_i > d_j) \quad (10)$$

where r_i is the Pareto front rank and d_i is the crowding distance.

Algorithm 2 NSGA-II Generational Replacement Loop

Require: Parent population $\mathcal{P}_t$ of size N , Offspring population $\mathcal{Q}_t$ of size N

Ensure: Next generation parent population $\mathcal{P}_{t+1}$ of size N

- 1: Combine populations: $\mathcal{R}_t \leftarrow \mathcal{P}_t \cup \mathcal{Q}_t$ ▷ Size $2N$
 - 2: Compute Pareto fronts: $\mathcal{F} \leftarrow \text{FastNonDominatedSort}(\mathcal{R}_t)$
 - 3: $\mathcal{P}_{t+1} \leftarrow \emptyset, i \leftarrow 1$
 - 4: **while** $|\mathcal{P}_{t+1}| + |\mathcal{F}_i| \leq N$ **do**
 - 5: ComputeCrowdingDistance($\mathcal{F}_i$)
 - 6: $\mathcal{P}_{t+1} \leftarrow \mathcal{P}_{t+1} \cup \mathcal{F}_i$
 - 7: $i \leftarrow i + 1$
 - 8: **end while**
 - 9: **if** $|\mathcal{P}_{t+1}| < N$ **then**
 - 10: ComputeCrowdingDistance($\mathcal{F}_i$)
 - 11: Sort $\mathcal{F}_i$ in descending order of crowding distance d
 - 12: Add top $(N - |\mathcal{P}_{t+1}|)$ individuals of $\mathcal{F}_i$ to $\mathcal{P}_{t+1}$
 - 13: **end if**
 - 14: **return** $\mathcal{P}_{t+1}$
-